

0.29.3. La fonction `chaque()` — toutes les `n` millisecondes

`#include <chaque> : if (chaque(500)) { ... } se fait deux fois par seconde, sans rien arrêter.`

1. Ce qu'il faut comprendre

Ce qu'elle fait : `chaque(ms)` répond **1** toutes les `ms` millisecondes, et **0** le reste du temps. Dans un `if`, le bloc se fait à ce rythme, pendant que le reste du jeu continue. Chaque `chaque()` du programme a son propre chronomètre (huit au plus).

La ligne à écrire : `#include <chaque>`.

Son argument : la durée en millisecondes, écrite en clair (elle est traduite en images avant le jeu).

Ce qu'elle coûte : environ **140 octets** la première fois : la petite fonction qui compte, et trois octets de mémoire par chronomètre.

Essaie : `chaque(100)` : le mot clignote vite.

On retrouve `chaque` dans 62 leçons

[0.77](#), [0.78](#), [0.78.1](#), [0.79](#), [0.79.1](#), [0.82](#), [0.82.1](#), [0.83.2](#), [0.84](#), [0.85](#), [0.88.2](#), [0.90.5](#), [0.90.6](#), [0.90.7](#), [0.90.8](#), [0.91.1](#), [0.91.2](#), [0.91.3](#), [0.91.4](#), [0.91.5](#), [0.91.6](#), [0.91.7](#), [0.91.8](#), [0.91.9](#), [0.92](#), [0.92.1](#), [0.92.2](#), [0.93.1](#), [0.93.2](#), [0.93.3](#), [0.93.4](#), [0.93.5](#), [0.93.6](#), [0.93.7](#), [0.93.8](#), [0.93.9](#), [0.94](#), [0.94.1](#), [0.95](#), [0.95.1](#), [0.96](#), [0.96.1](#), [0.97](#), [0.97.1](#), [0.98](#), [0.98.1](#), [0.99](#), [0.99.1](#), [0.100](#), [0.100.1](#), [0.101](#), [0.101.1](#), [0.102](#), [0.102.1](#), [0.103](#), [0.104](#), [0.105](#), [0.105.1](#), [0.106](#), [0.107](#), [0.107.1](#), [0.108](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **659 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	52 o	← la console, à l'allumage	— la mise en route de la console
chaque_minuteur()	124 o	← main()	— une fonction fournie par le langage
main()	67 o	← démarrage	— ta fonction
VBlank	8 o	← la console, à chaque image	— compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	— attend l'image suivante
AttendreVBlank	16 o	← démarrage	— attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte	— attend de pouvoir écrire une case
EffacerCases	12 o	← effacer()	— efface des cases
EcrireTexte	10 o	← texte()	— écrit un texte, lettre par lettre
CopierTuiles	121 o	← démarrage	— copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	— vide les deux cartes de l'écran
Tuiles	176 o	← CopierTuiles	— 11 lettres sur 43
"CHAQUE 500 MS"	13 o	← texte()	— un texte gravé
"COUCOU"	6 o	← texte()	— un texte gravé

Lettres gravées : ACEHMOQSU05

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,

DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 2	22 o sur place, dans main()	+ EcrireTexte (10 o)
effacer()	× 1	8 o sur place, dans main()	+ EffacerCases (12 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()	

Tes fonctions appelées :

chaque_minuteur()	× 1	10 o sur place, dans main()
-------------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <chaque> : toutes les n millisecondes ----
// Toutes les 500 ms, le mot apparaît ou disparaît : il clignote.

#include <chaque>    // chaque() : 1 toutes les n millisecondes
#include <texte>
#include <effacer>
```

2. UNE VARIABLE — ALLUME

```
uint8_t allume = 0;
```

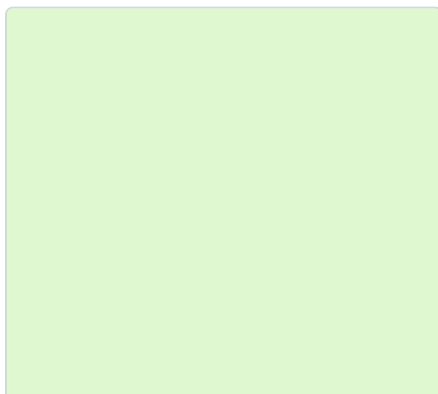
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(4, 2, "CHAQUE 500 MS");    // un titre fixe, qui ne clignote pas

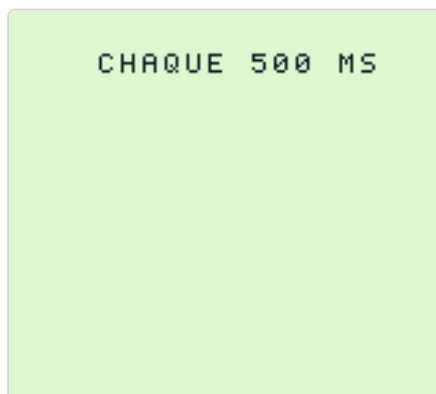
    while (true) {
        image();
        if (chaque(500)) {           // deux fois par seconde
            if (allume == 0) {
                texte(6, 8, "COUCOU");
                allume = 1;
            } else {
                effacer(6, 8, "COUCOU");
                allume = 0;
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

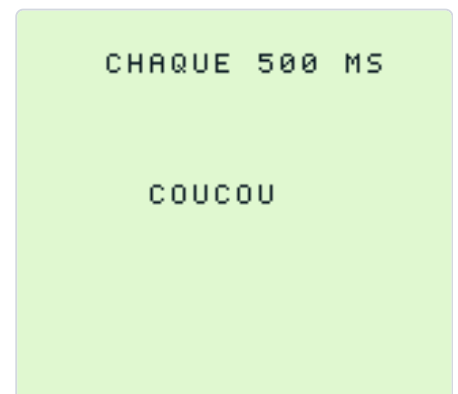
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60

4. Ce qu'on doit voir

CHAQUE 500 MS en haut, et COUCOU qui clignote en dessous, deux fois par seconde.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le mot apparaît

✓ et disparaît

Ensuite : **0.29.4. La fonction `un_pas()` — un seul pas, sans attendre** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++